CPU Scheduling Simulator

Visual Comparison of FCFS, SJF, SRTF, Priority and Round Robin Algorithms

A Mini Project Report

Subject: Operating Systems

Submitted by	Vishwaa SJ
Register No.	312424205254
Department	Information Technology, B.Tech - III Year
Guide	Mr. Abhijith
Institution	St. Joseph's Institute of Technology
Academic Year	2026 - 2027

Abstract

CPU scheduling decides which process in the ready queue gets the processor and for how long. The choice of algorithm directly affects waiting time, turnaround time, response time and CPU utilization. This project implements a simulator in Python for five classical scheduling algorithms - First Come First Serve (FCFS), Shortest Job First (SJF), Shortest Remaining Time First (SRTF), Priority Scheduling and Round Robin (RR). Given a set of processes, the tool computes per-process metrics, prints a Gantt chart, produces comparison graphs and identifies the best algorithm for the workload. The implementation is validated with unit tests against hand-calculated textbook results.

1. Introduction

In a multiprogramming operating system several processes compete for a single CPU. The short-term scheduler selects the next process from the ready queue. Understanding the trade-offs between algorithms is a core OS topic, but hand-drawing Gantt charts is slow and error-prone. A simulator makes it possible to experiment with different workloads instantly.

1.1 Objectives

- Implement five scheduling algorithms with correct handling of arrival times and CPU idle gaps.
- Calculate Completion, Turnaround, Waiting and Response time for each process.
- Generate Gantt charts and comparison graphs automatically.
- Verify correctness through automated unit tests.
- Allow custom workloads through a CSV file and a configurable time quantum.

1.2 Scope and Tools

Language: Python 3. Libraries: *matplotlib* and *numpy* for charts, *reportlab* for this report; the core scheduling logic uses only the standard library (*dataclasses*, *collections.deque*, *unittest*).

2. Algorithms Implemented

Algorithm	Type	Selection rule	Pros / Cons
FCFS	Non-preemptive	Earliest arrival	Simple; convoy effect
SJF	Non-preemptive	Smallest burst time	Low avg. wait; starvation
SRTF	Preemptive	Smallest remaining time	Optimal avg. wait; many switches
Priority	Non-preemptive	Lowest priority number	Flexible; starvation
Round Robin	Preemptive	FIFO with time quantum	Fair, good response; high TAT

2.1 Performance Metrics

- **Turnaround Time (TAT)** = Completion Time - Arrival Time
- **Waiting Time (WT)** = Turnaround Time - Burst Time
- **Response Time (RT)** = First CPU time - Arrival Time
- **CPU Utilization** = (Busy time / Total time) x 100
- **Throughput** = Number of processes / Total time

3. System Design and Implementation

The project is split into three modules so that logic, interface and verification stay independent.

File	Responsibility
scheduler.py	Process and Result data classes, the five algorithms, input validation
main.py	Command-line interface, CSV loader, text tables, Gantt and comparison plots
test_scheduler.py	8 unit tests checking waiting times, idle gaps and error handling
build_report.py	Generates this PDF report

3.1 Key Design Decisions

- **Shared engine for non-preemptive algorithms:** FCFS, SJF and Priority differ only in the sort key, so one function takes the key as a parameter - this removes duplicate code.
- **Idle handling:** If no process has arrived, the clock jumps to the next arrival and an IDLE segment is recorded.
- **SRTF** is simulated one time unit at a time so that preemption on every new arrival is automatic.
- **Round Robin** enqueues processes that arrive during a time slice *before* re-queuing the preempted process, matching the standard textbook convention.
- **Gantt merging:** consecutive slices of the same process are merged into a single block.
- **Input validation:** bursts must be positive, IDs unique, quantum greater than zero.

4. Results and Analysis

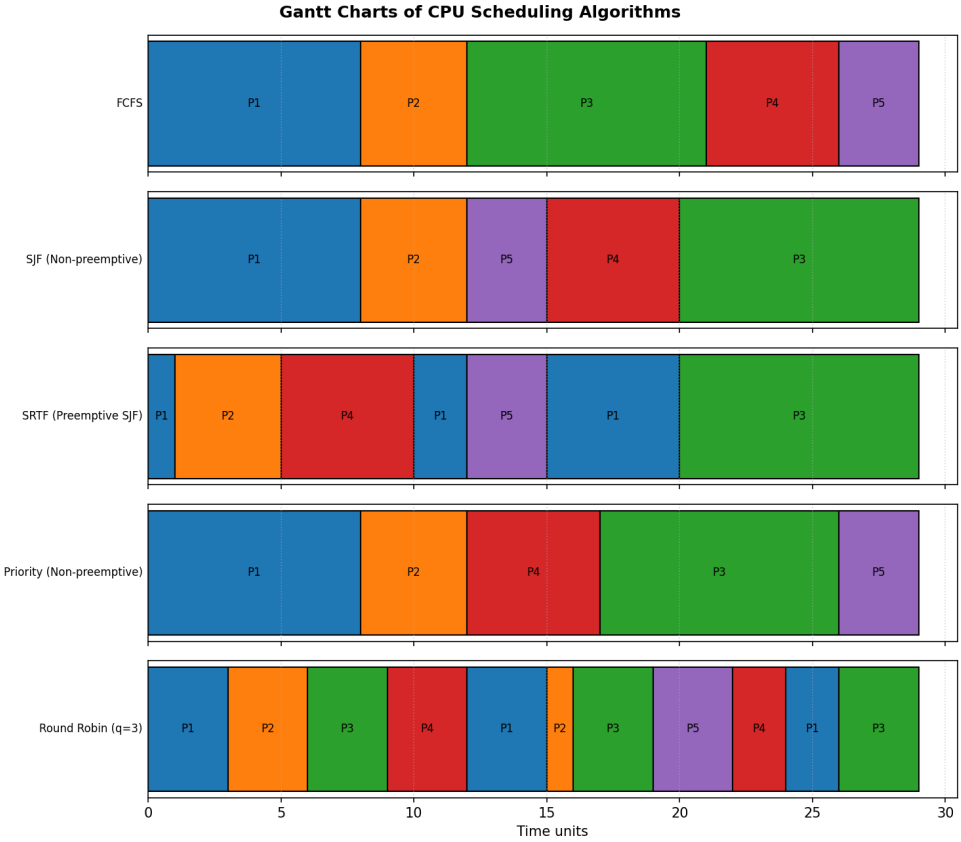
4.1 Input Workload

Process	Arrival	Burst	Priority
P1	0	8	3
P2	1	4	1
P3	2	9	4
P4	3	5	2
P5	12	3	5

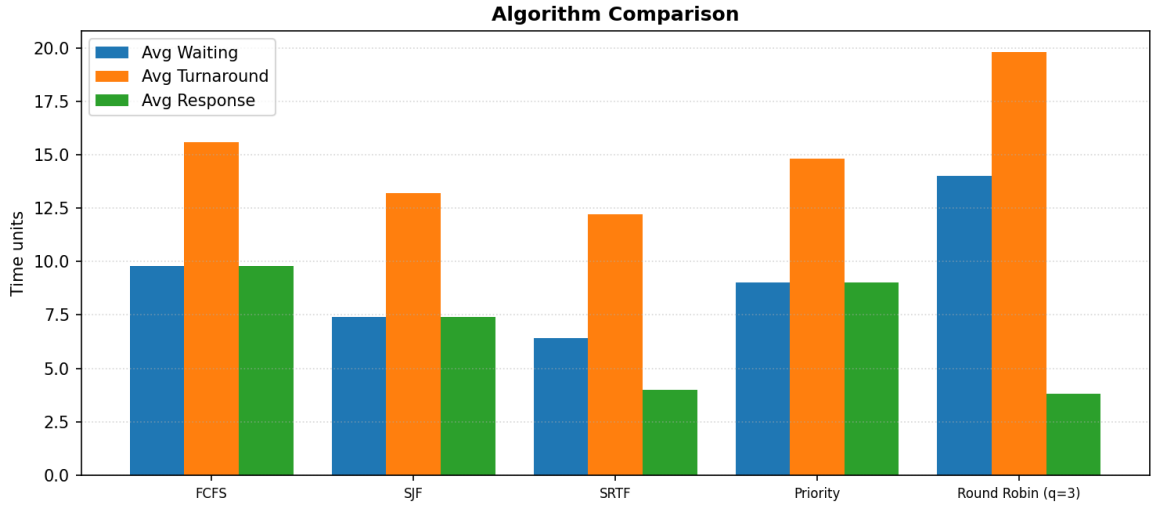
4.2 Summary of Metrics

Algorithm	Avg TAT	Avg WT	Avg RT	CPU Util.	Context switches
FCFS	15.60	9.80	9.80	100%	4
SJF (Non-preemptive)	13.20	7.40	7.40	100%	4
SRTF (Preemptive SJF)	12.20	6.40	4.00	100%	6
Priority (Non-preemptive)	14.80	9.00	9.00	100%	4
Round Robin (q=3)	19.80	14.00	3.80	100%	10

4.3 Gantt Charts



4.4 Algorithm Comparison



4.5 Observations

- **SRTF (Preemptive SJF)** gave the lowest average waiting time (6.40), confirming that SRTF is provably optimal for average waiting time - at the cost of more context switches.
- **Round Robin** had the best fairness - every process received the CPU early (average response time 3.80) - but the highest average turnaround (19.80) because long jobs are repeatedly interrupted.
- **FCFS** suffers from the convoy effect: P3 (burst 9) delays all shorter jobs behind it.
- **Priority** scheduling favoured the urgent process P2 and P4, but P5 (lowest priority) waited until the end - an illustration of starvation risk.
- No single algorithm is best in all metrics; the right choice depends on whether the system values throughput, fairness or responsiveness (e.g. batch vs. interactive systems).

5. Testing

The simulator is verified with a classic textbook workload (P1: 0/7, P2: 2/4, P3: 4/1, P4: 5/4 as arrival/burst). Expected waiting times were calculated by hand and compared with program output.

Test	What it checks	Result
test_fcfs	WT = {0, 5, 7, 7}	Pass
test_sjf	WT = {0, 6, 3, 7}	Pass
test_srtf	WT = {9, 1, 0, 2}, avg 3.0	Pass
test_round_robin	Completion time = total burst	Pass
test_priority	Highest priority picked first	Pass
test_idle_gap	IDLE block and 57% utilization	Pass
test_invalid	Errors for burst 0 / quantum 0	Pass
test_metrics_identity	TAT = WT + BT for all	Pass

6. Conclusion

The simulator reproduces textbook scheduling behaviour accurately and makes the differences between algorithms visible through Gantt charts and comparison graphs. The project strengthened understanding of preemption, starvation, the convoy effect and the fairness-versus-efficiency trade-off.

7. Future Enhancements

- Multilevel Queue and Multilevel Feedback Queue scheduling.
- Preemptive priority scheduling with aging to prevent starvation.
- Context-switch overhead and I/O burst modelling.
- A web or Tkinter GUI for entering processes interactively.
- Multi-core scheduling simulation.

8. How to Run

```
pip install matplotlib numpy
python main.py                   # run with built-in sample
python main.py --quantum 4       # change RR quantum
python main.py --file sample_processes.csv
python -m unittest -v            # run tests
```

9. References

- Silberschatz, Galvin, Gagne - *Operating System Concepts*, 10th Edition, Wiley.
- Tanenbaum, A. S. - *Modern Operating Systems*, 4th Edition, Pearson.
- Python Documentation - https://docs.python.org/3/
- Matplotlib Documentation - https://matplotlib.org/

Appendix A: Source Code - scheduler.py

```
"""
scheduler.py - Core CPU scheduling algorithms.

Each algorithm takes a list of Process objects and returns a Result containing
the Gantt chart (list of (pid, start, end) segments) and per-process metrics.
Lower priority number = higher priority.
"""
from dataclasses import dataclass, field
from collections import deque
from typing import List, Tuple

IDLE = "IDLE"


@dataclass
class Process:
    pid: str
    arrival: int
    burst: int
    priority: int = 0
    # filled in by the simulator
    remaining: int = field(init=False)
    start: int = field(default=-1, init=False)
    completion: int = field(default=0, init=False)

    def __post_init__(self):
        self.remaining = self.burst

    @property
    def turnaround(self) -> int:
        return self.completion - self.arrival

    @property
    def waiting(self) -> int:
        return self.turnaround - self.burst

    @property
    def response(self) -> int:
        return self.start - self.arrival


@dataclass
class Result:
    name: str
    gantt: List[Tuple[str, int, int]]
    processes: List[Process]

    def avg(self, attr: str) -> float:
        return sum(getattr(p, attr) for p in self.processes) / len(self.processes)

    @property
    def cpu_utilization(self) -> float:
        busy = sum(e - s for pid, s, e in self.gantt if pid != IDLE)
        total = self.gantt[-1][2] - self.gantt[0][1]
        return 100.0 * busy / total if total else 0.0

    @property
    def throughput(self) -> float:
        total = max(p.completion for p in self.processes) - min(p.arrival for p in self.processes)
        return len(self.processes) / total if total else 0.0


def _clone(procs: List[Process]) -> List[Process]:
    return [Process(p.pid, p.arrival, p.burst, p.priority) for p in procs]


def _add(gantt, pid, start, end):
    """Append a segment, merging with the previous one if it is the same pid."""
    if end <= start:
        return
    if gantt and gantt[-1][0] == pid and gantt[-1][2] == start:
        gantt[-1] = (pid, gantt[-1][1], end)
    else:
        gantt.append((pid, start, end))


def _validate(procs):
    if not procs:
        raise ValueError("Process list is empty")
    for p in procs:
        if p.burst <= 0 or p.arrival < 0:
            raise ValueError(f"Invalid process {p.pid}: burst must be > 0 and arrival >= 0")
    if len({p.pid for p in procs}) != len(procs):
```

```
        raise ValueError("Duplicate process IDs")


# ---------------------------------------------------------------------------
# Non-preemptive algorithms (FCFS, SJF, Priority share one engine)
# ---------------------------------------------------------------------------
def _non_preemptive(name, procs, key):
    _validate(procs)
    ps = _clone(procs)
    gantt, time, done = [], 0, 0
    while done < len(ps):
        ready = [p for p in ps if p.arrival <= time and p.remaining > 0]
        if not ready:
            nxt = min(p.arrival for p in ps if p.remaining > 0)
            _add(gantt, IDLE, time, nxt)
            time = nxt
            continue
        p = min(ready, key=key)
        p.start = time
        _add(gantt, p.pid, time, time + p.burst)
        time += p.burst
        p.remaining = 0
        p.completion = time
        done += 1
    return Result(name, gantt, ps)


def fcfs(procs):
    return _non_preemptive("FCFS", procs, key=lambda p: (p.arrival, p.pid))


def sjf(procs):
    return _non_preemptive("SJF (Non-preemptive)", procs, key=lambda p: (p.burst, p.arrival, p.pid))


def priority_np(procs):
    return _non_preemptive("Priority (Non-preemptive)", procs, key=lambda p: (p.priority, p.arrival, p.pid))


# ---------------------------------------------------------------------------
# Preemptive: SRTF (Shortest Remaining Time First)
# ---------------------------------------------------------------------------
def srtf(procs):
    _validate(procs)
    ps = _clone(procs)
    gantt, time, done = [], 0, 0
    while done < len(ps):
        ready = [p for p in ps if p.arrival <= time and p.remaining > 0]
        if not ready:
            nxt = min(p.arrival for p in ps if p.remaining > 0)
            _add(gantt, IDLE, time, nxt)
            time = nxt
            continue
        p = min(ready, key=lambda x: (x.remaining, x.arrival, x.pid))
        if p.start == -1:
            p.start = time
        _add(gantt, p.pid, time, time + 1)   # run one time unit
        p.remaining -= 1
        time += 1
        if p.remaining == 0:
            p.completion = time
            done += 1
    return Result("SRTF (Preemptive SJF)", gantt, ps)


# ---------------------------------------------------------------------------
# Round Robin
# ---------------------------------------------------------------------------
def round_robin(procs, quantum=2):
    _validate(procs)
    if quantum <= 0:
        raise ValueError("Quantum must be > 0")
    ps = sorted(_clone(procs), key=lambda p: (p.arrival, p.pid))
    gantt, time, done, i = [], 0, 0, 0
    queue = deque()
    while done < len(ps):
        while i < len(ps) and ps[i].arrival <= time:
            queue.append(ps[i]); i += 1
        if not queue:
            nxt = ps[i].arrival
            _add(gantt, IDLE, time, nxt)
            time = nxt
            continue
        p = queue.popleft()
        if p.start == -1:
            p.start = time
        run = min(quantum, p.remaining)
```

```
        _add(gantt, p.pid, time, time + run)
        time += run
        p.remaining -= run
        # processes that arrived while p was running go in BEFORE p is requeued
        while i < len(ps) and ps[i].arrival <= time:
            queue.append(ps[i]); i += 1
        if p.remaining == 0:
            p.completion = time
            done += 1
        else:
            queue.append(p)
    return Result(f"Round Robin (q={quantum})", gantt, ps)


ALGORITHMS = {
    "fcfs": fcfs,
    "sjf": sjf,
    "srtf": srtf,
    "priority": priority_np,
    "rr": round_robin,
}
```

Appendix B: Source Code - main.py

```
"""
main.py - CLI + Gantt chart generator for the CPU Scheduling Simulator.

Usage:
    python main.py                # run all algorithms on the built-in sample
    python main.py --quantum 3    # change Round Robin quantum
    python main.py --file procs.csv  # CSV columns: pid,arrival,burst,priority
    python main.py --no-plot      # text output only
"""
import argparse
import csv
import os

from scheduler import Process, Result, IDLE, fcfs, sjf, srtf, priority_np, round_robin

SAMPLE = [
    Process("P1", 0, 8, 3),
    Process("P2", 1, 4, 1),
    Process("P3", 2, 9, 4),
    Process("P4", 3, 5, 2),
    Process("P5", 12, 3, 5),   # arrives later, after the first batch
]


def load_csv(path):
    procs = []
    with open(path, newline="") as f:
        for row in csv.DictReader(f):
            procs.append(Process(row["pid"].strip(), int(row["arrival"]),
                                 int(row["burst"]), int(row.get("priority", 0) or 0)))
    return procs


def print_result(r: Result):
    print(f"\n=== {r.name} ===")
    print(f"{'PID':<5}{'AT':>4}{'BT':>4}{'PR':>4}{'CT':>5}{'TAT':>5}{'WT':>5}{'RT':>5}")
    for p in sorted(r.processes, key=lambda p: p.pid):
        print(f"{p.pid:<5}{p.arrival:>4}{p.burst:>4}{p.priority:>4}"
              f"{p.completion:>5}{p.turnaround:>5}{p.waiting:>5}{p.response:>5}")
    print(f"Avg TAT = {r.avg('turnaround'):.2f} | Avg WT = {r.avg('waiting'):.2f} | "
          f"Avg RT = {r.avg('response'):.2f}")
    print(f"CPU Utilization = {r.cpu_utilization:.1f}% | Throughput = {r.throughput:.3f} proc/unit")
    print("Gantt: " + " ".join(f"[{s}| {pid} |{e}]" for pid, s, e in r.gantt))


def plot_gantt(results, outdir):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    pids = sorted({p.pid for r in results for p in r.processes})
    cmap = plt.get_cmap("tab10")
    colors = {pid: cmap(i % 10) for i, pid in enumerate(pids)}
    colors[IDLE] = "#d9d9d9"

    # --- combined Gantt chart ---
    fig, axes = plt.subplots(len(results), 1, figsize=(10, 1.6 * len(results) + 0.8), sharex=True)
    for ax, r in zip(axes, results):
        for pid, s, e in r.gantt:
            ax.barh(0, e - s, left=s, color=colors[pid], edgecolor="black", height=0.6)
            ax.text((s + e) / 2, 0, pid if pid != IDLE else "idle", ha="center", va="center", fontsize=8)
        ax.set_yticks([])
        ax.set_ylabel(r.name, rotation=0, ha="right", va="center", fontsize=8)
        ax.grid(axis="x", linestyle=":", alpha=0.5)
    axes[-1].set_xlabel("Time units")
    fig.suptitle("Gantt Charts of CPU Scheduling Algorithms", fontweight="bold")
    fig.tight_layout()
    gpath = os.path.join(outdir, "gantt_charts.png")
    fig.savefig(gpath, dpi=150)
    plt.close(fig)

    # --- comparison bar chart ---
    import numpy as np
    names = [r.name.split(" (")[0] if "Round" not in r.name else r.name for r in results]
    x = np.arange(len(results)); w = 0.27
    fig, ax = plt.subplots(figsize=(10, 4.5))
    ax.bar(x - w, [r.avg("waiting") for r in results], w, label="Avg Waiting")
    ax.bar(x, [r.avg("turnaround") for r in results], w, label="Avg Turnaround")
    ax.bar(x + w, [r.avg("response") for r in results], w, label="Avg Response")
    ax.set_xticks(x); ax.set_xticklabels(names, fontsize=8)
    ax.set_ylabel("Time units"); ax.set_title("Algorithm Comparison", fontweight="bold")
    ax.legend(); ax.grid(axis="y", linestyle=":", alpha=0.5)
    fig.tight_layout()
```

```
    cpath = os.path.join(outdir, "comparison.png")
    fig.savefig(cpath, dpi=150)
    plt.close(fig)
    return gpath, cpath


def main():
    ap = argparse.ArgumentParser(description="CPU Scheduling Simulator")
    ap.add_argument("--file", help="CSV file with columns pid,arrival,burst,priority")
    ap.add_argument("--quantum", type=int, default=3, help="Round Robin time quantum")
    ap.add_argument("--outdir", default="output", help="Where charts are saved")
    ap.add_argument("--no-plot", action="store_true")
    args = ap.parse_args()

    procs = load_csv(args.file) if args.file else SAMPLE
    results = [fcfs(procs), sjf(procs), srtf(procs), priority_np(procs),
               round_robin(procs, args.quantum)]
    for r in results:
        print_result(r)

    best = min(results, key=lambda r: r.avg("waiting"))
    print(f"\n>>> Lowest average waiting time: {best.name} ({best.avg('waiting'):.2f})")

    if not args.no_plot:
        os.makedirs(args.outdir, exist_ok=True)
        g, c = plot_gantt(results, args.outdir)
        print(f"Charts saved: {g}, {c}")


if __name__ == "__main__":
    main()
```

Appendix C: Source Code - test_scheduler.py

```
import unittest
from scheduler import Process, fcfs, sjf, srtf, priority_np, round_robin, IDLE


def textbook():
    return [Process("P1", 0, 7, 2), Process("P2", 2, 4, 1),
            Process("P3", 4, 1, 3), Process("P4", 5, 4, 2)]


class TestSchedulers(unittest.TestCase):
    def wt(self, r):
        return {p.pid: p.waiting for p in r.processes}

    def test_fcfs(self):
        r = fcfs(textbook())
        self.assertEqual(self.wt(r), {"P1": 0, "P2": 5, "P3": 7, "P4": 7})

    def test_sjf(self):
        r = sjf(textbook())
        self.assertEqual(self.wt(r), {"P1": 0, "P2": 6, "P3": 3, "P4": 7})

    def test_srtf(self):
        r = srtf(textbook())
        self.assertEqual(self.wt(r), {"P1": 9, "P2": 1, "P3": 0, "P4": 2})
        self.assertAlmostEqual(r.avg("waiting"), 3.0)

    def test_round_robin(self):
        r = round_robin(textbook(), 2)
        self.assertEqual(sum(p.burst for p in r.processes), 16)
        self.assertEqual(max(p.completion for p in r.processes), 16)

    def test_priority(self):
        r = priority_np(textbook())
        self.assertEqual(r.gantt[0][0], "P1")      # only one ready at t=0
        self.assertEqual(r.gantt[1][0], "P2")      # priority 1 beats others

    def test_idle_gap(self):
        r = fcfs([Process("A", 0, 2), Process("B", 5, 2)])
        self.assertIn((IDLE, 2, 5), r.gantt)
        self.assertAlmostEqual(r.cpu_utilization, 4 / 7 * 100)

    def test_invalid(self):
        with self.assertRaises(ValueError):
            fcfs([Process("A", 0, 0)])
        with self.assertRaises(ValueError):
            round_robin(textbook(), 0)

    def test_metrics_identity(self):
        for algo in (fcfs, sjf, srtf, priority_np):
            for p in algo(textbook()).processes:
                self.assertEqual(p.turnaround, p.waiting + p.burst)


if __name__ == "__main__":
    unittest.main()
```

Appendix D: Console Output

```
=== FCFS ===
PID    AT  BT  PR   CT  TAT   WT   RT
P1      0   8   3    8    8    0    0
P2      1   4   1   12   11    7    7
P3      2   9   4   21   19   10   10
P4      3   5   2   26   23   18   18
P5     12   3   5   29   17   14   14
Avg TAT = 15.60 | Avg WT = 9.80 | Avg RT = 9.80
CPU Utilization = 100.0% | Throughput = 0.172 proc/unit
Gantt: [0| P1 |8] [8| P2 |12] [12| P3 |21] [21| P4 |26] [26| P5 |29]

=== SJF (Non-preemptive) ===
PID    AT  BT  PR   CT  TAT   WT   RT
P1      0   8   3    8    8    0    0
P2      1   4   1   12   11    7    7
P3      2   9   4   29   27   18   18
P4      3   5   2   20   17   12   12
P5     12   3   5   15    3    0    0
Avg TAT = 13.20 | Avg WT = 7.40 | Avg RT = 7.40
CPU Utilization = 100.0% | Throughput = 0.172 proc/unit
Gantt: [0| P1 |8] [8| P2 |12] [12| P5 |15] [15| P4 |20] [20| P3 |29]

=== SRTF (Preemptive SJF) ===
PID    AT  BT  PR   CT  TAT   WT   RT
P1      0   8   3   20   20   12    0
P2      1   4   1    5    4    0    0
P3      2   9   4   29   27   18   18
P4      3   5   2   10    7    2    2
P5     12   3   5   15    3    0    0
Avg TAT = 12.20 | Avg WT = 6.40 | Avg RT = 4.00
CPU Utilization = 100.0% | Throughput = 0.172 proc/unit
Gantt: [0| P1 |1] [1| P2 |5] [5| P4 |10] [10| P1 |12] [12| P5 |15] [15| P1 |20] [20| P3 |29]

=== Priority (Non-preemptive) ===
PID    AT  BT  PR   CT  TAT   WT   RT
P1      0   8   3    8    8    0    0
P2      1   4   1   12   11    7    7
P3      2   9   4   26   24   15   15
P4      3   5   2   17   14    9    9
P5     12   3   5   29   17   14   14
Avg TAT = 14.80 | Avg WT = 9.00 | Avg RT = 9.00
CPU Utilization = 100.0% | Throughput = 0.172 proc/unit
Gantt: [0| P1 |8] [8| P2 |12] [12| P4 |17] [17| P3 |26] [26| P5 |29]

=== Round Robin (q=3) ===
PID    AT  BT  PR   CT  TAT   WT   RT
P1      0   8   3   26   26   18    0
P2      1   4   1   16   15   11    2
P3      2   9   4   29   27   18    4
P4      3   5   2   24   21   16    6
P5     12   3   5   22   10    7    7
Avg TAT = 19.80 | Avg WT = 14.00 | Avg RT = 3.80
CPU Utilization = 100.0% | Throughput = 0.172 proc/unit
Gantt: [0| P1 |3] [3| P2 |6] [6| P3 |9] [9| P4 |12] [12| P1 |15] [15| P2 |16] [16| P3 |19] [19| P5 |22] [22| P4 |24] [24| P1 |26]

>>> Lowest average waiting time: SRTF (Preemptive SJF) (6.40)
Charts saved: output/gantt_charts.png, output/comparison.png
```